Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

All of the drafter things with buttons, pages, indexes, classes, and lists. In addition to the loops and conditionals.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The hardest will be the visualization of everything I want to do as we haven't really learned about animation. The easiest will be all the classes and functions to get to the visual aspect.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Most time would be the set up and data for each scenario and making sure the type of road and the type of interchange work together.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Apart from the obvious in my pocket, there will be other distractions like other school work and friends. I will be using an LLM for my coding so that I don't have to figure everything out myself.

1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

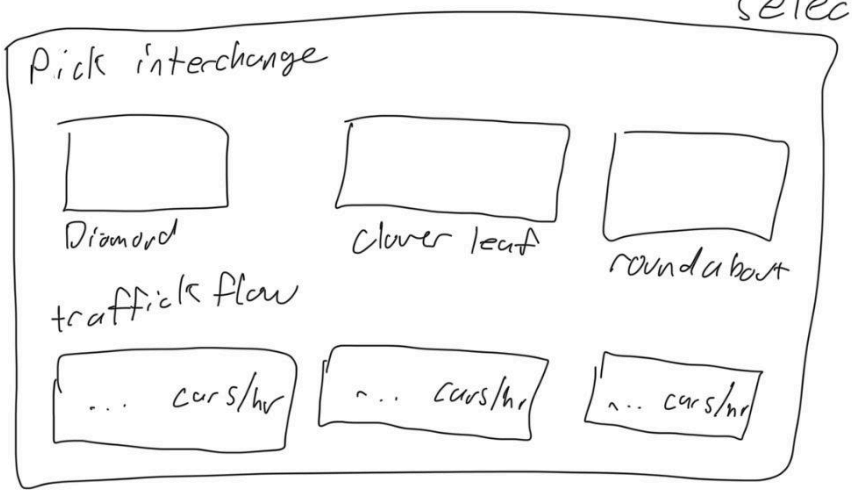
In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website. add photo(s) or screenshot(s) of your website mock-up and labeled

diagram on the next page.

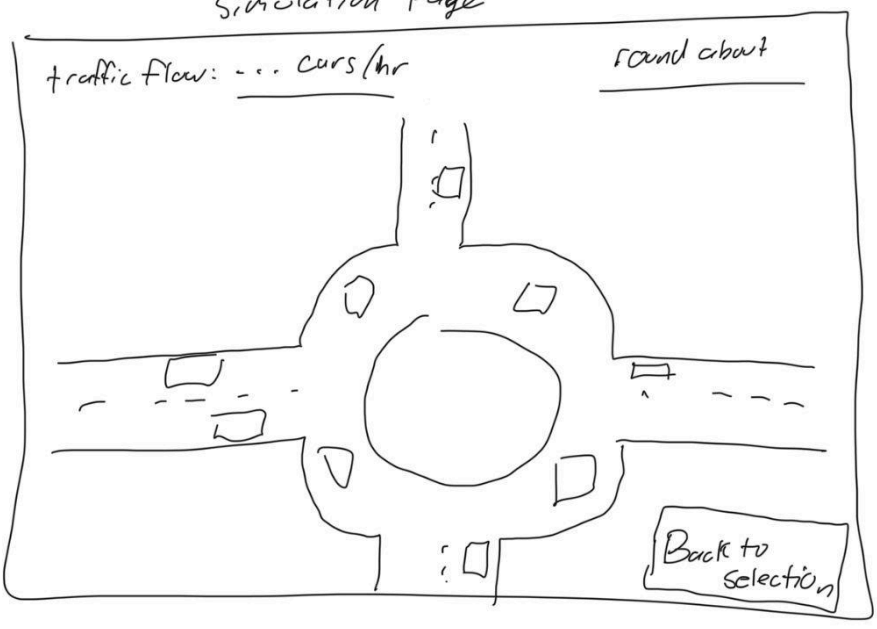
11:03 AM Wed Nov 4
89%
Aa
Suggested Title: Final Project 1 Design: Edit
November 4, 2026 at 11:01 AM

Final Project 1 Design:

Selection Page



Simulation Page



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	The purpose of this program will be to better visualize/understand traffic flow through different intersections depending on how dense the traffic is.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	Intersection_type: str - This stores the type of intersection and changes when the user switches to a different intersection. Traffic_density: int - The amount of traffic, 1(low), 2(med), 3(high). Vehicles: list[Vehicle] - Stores all of the vehicles in the simulation. Simulation_running: bool - States whether the simulation is running or not Simulation_step: int - Tells how many simulation steps have happened
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Class Vehicle: Vehicle_id: int - Individual identification number Position: int - A number that represents the vehicles position on the route Speed: int - How far the vehicle can move during each

	simulation step Direction: str - The vehicles direction of travel Waiting: bool - Tells whether the vehicle is stopped in traffic
Routes (minimum 7) The names of your route functions	Home: homepage Choose_intersection: intersection selection page Set_traffic: traffic density selection Simulation: traffic visualization in current intersection Start_simulation: starts the simulation with vehicles Advance_simulation: moves vehicles Reset_simulation: resets simulation Results: statistics and simulation results
Pages (minimum 5) A brief description of the pages in your site	Homepage: purpose of program and shows how people can explore traffic flow. Intersection selection: where the user can select which intersection they want to observe Traffic settings: where users can select the density of traffic, and vehicle speed that they want to observe. Simulation: the simulation visual of the intersection. Results: where the statistics and simulation results are
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	Selectbox - intersection type: choose which type of intersection the user wants to observe selectBox - traffic density: select between low, medium, and high SelectBox - vehicle speed: select between slow, normal, and fast speeds

If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Traffic Density: checks the selection of traffic density and determines how many vehicles will be generated. Intersection type: checks what intersection was chosen and determines which layout and vehicle movements to use. Vehicle congestion: checks if path is blocked and stops/slows vehicles to allow congestion to develop. Simulation Status: Checks if simulation is running and tells vehicles to advance.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	Iterates over the vehicles list in the state dataclass and updates the positions of every vehicle during each simulation step.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

for the conversation called traffic tool features, I am moving forward with the project. and have some requirements that I need

Sounds good! We can continue building on the Traffic Flow Visualization Tool project from our previous conversation.

From our discussion, your idea was to create a Python-based web application using Drafter that allows users to visualize how traffic flows through different road types and interchange designs.

The main focus was to help users understand how different interchange layouts affect traffic movement, congestion, and efficiency.

As you move forward, I can help you with:

- Project requirements: Making sure your application meets the assignment's specifications.
- Program design: Planning the functions, data structures, and overall organization.
- Visualizations: Figuring out how to display roads, interchanges, and moving traffic using Python and Drafter.
- Implementation: Building the program step by step while keeping the code manageable.
- Testing: Making sure the application works correctly and meets your requirements.

Send me the requirements you have, and we'll work through them while keeping your

original project idea in mind.

here is the purpose of my program, The purpose of this program will be to better visualize/understand traffic flow through different intersections depending on how dense the traffic is.

That's a solid starting point for your project. I would slightly refine the wording to make the purpose clearer and more specific.

The purpose of this program is to help users visualize and better understand how traffic flows through different types of intersections under varying levels of traffic density. By comparing intersection designs and traffic conditions, users can observe how these factors affect traffic flow and congestion.

For your Python and Drafter project, this gives us three main objectives:

1. Intersection selection: Allow users to choose between different intersection designs.
2. Traffic density: Allow users to adjust the amount of traffic (low, medium, or high).
3. Traffic visualization: Display how vehicles move through the selected intersection and demonstrate how congestion develops.

One important distinction is whether your program will simulate moving vehicles or simply display visual representations of traffic flow. Both could accomplish your purpose, but they would require different levels of programming complexity.

Go ahead and send me your next requirement, and we can build the project around these objectives.

these are the requirments,

State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.

Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)

Routes (minimum 7)

The names of your route functions

Pages (minimum 5)

A brief description of the pages in your site

User Input (minimum 3)

Description of user input components (TextBox, TextArea,

CheckBox, or SelectBox) and their purposes

For more information on these input types, see .

If Statements (minimum 3)

Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site

---

Loop (minimum 1)

This loop must iterate over the list/dict of dataclass instances in your State attribute.

Description of this loop and why it is meaningful to the site

give me what I should do for all of these based on my vision.

...

Traffic Flow Visualization Tool — Project Plan

Based on your vision, I would design your project as an interactive traffic intersection simulator using Python and Drafter.

The user would be able to select different types of intersections, adjust traffic density, and visualize how traffic moves through each design.

Since this is a programming project, I recommend keeping the simulation relatively simple while making sure every feature contributes to your overall purpose.

1. State Dataclass Attributes (minimum 4)

Your `State` dataclass will store information about the current simulation and the user's selections.

I recommend 5 attributes:

Attribute	Type	Purpose and how it changes
`intersection_type`	`str`	Stores the selected intersection design. Changes when the user selects a different intersection.
`traffic_density`	`int`	Stores the amount of traffic, such as 1 (low), 2 (medium), or 3 (high). Changes when the user adjusts traffic density.
`vehicles`	`list[Vehicle]`	Stores all vehicles in the simulation. Vehicles are added, moved, or removed as the simulation progresses.

`simulation_running`	`bool`	Tracks whether the simulation is running or paused. Changes when the user starts or pauses the simulation.
`simulation_step`	`int`	Tracks how many simulation steps have occurred. Increases whenever the simulation advances and resets when restarted.

The most important attribute is `vehicles`, because it satisfies the requirement for a list containing instances of your second dataclass.

2. Second Dataclass

Your second dataclass should be called `Vehicle`.

Each vehicle will have information that determines how it moves through an intersection.

Attribute	Type	Purpose and how it changes
`vehicle_id`	`int`	Unique number identifying each vehicle. Stays the same.
`position`	`int`	Represents the vehicle's position along its route. Increases as it moves.
`speed`	`int`	Represents how far the vehicle moves during each simulation step. Can decrease when traffic becomes congested.
`direction`	`str`	Stores the vehicle's direction of travel (north, south, east, or west). Usually stays the same.
`waiting`	`bool`	Indicates whether a vehicle is stopped because of traffic. Changes based on congestion and intersection conditions.

Example Python structure:

```
from dataclasses import dataclass, field




@dataclass

class Vehicle:

    vehicle_id: int

    position: int

    speed: int

    direction: str

    waiting: bool
```

```python
@dataclass

class State:

    intersection_type: str

    traffic_density: int

    vehicles: list[Vehicle] = field(default_factory=list)

    simulation_running: bool = False

    simulation_step: int = 0
```

For your first version, vehicles could move forward in discrete steps whenever the user clicks an Advance Simulation button. This is easier to implement than continuous

animation.

3. Routes (minimum 7)

I recommend 8 routes so you have a little flexibility.

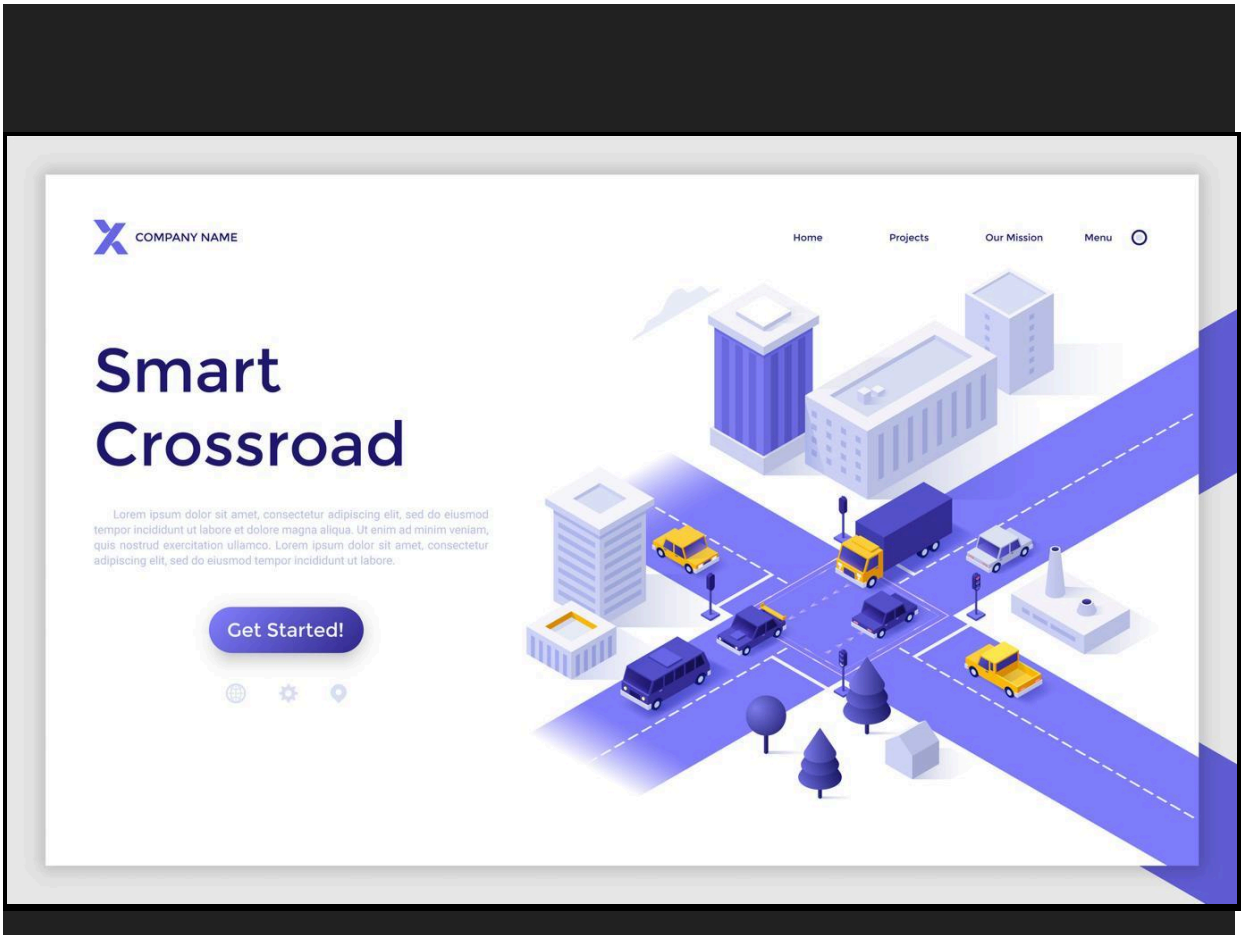
Route function	Purpose
`home`	Displays the homepage and project introduction.
`choose_intersection`	Displays the intersection selection page.
`set_traffic`	Processes the user's traffic density selection.
`simulation`	Displays the current intersection and traffic visualization.
`start_simulation`	Starts the simulation and creates vehicles.

`advance_simulation`	Moves vehicles forward and updates congestion.
`reset_simulation`	Clears vehicles and resets the simulation.
`results`	Displays traffic statistics and simulation results.

These routes would handle navigation and user interactions.

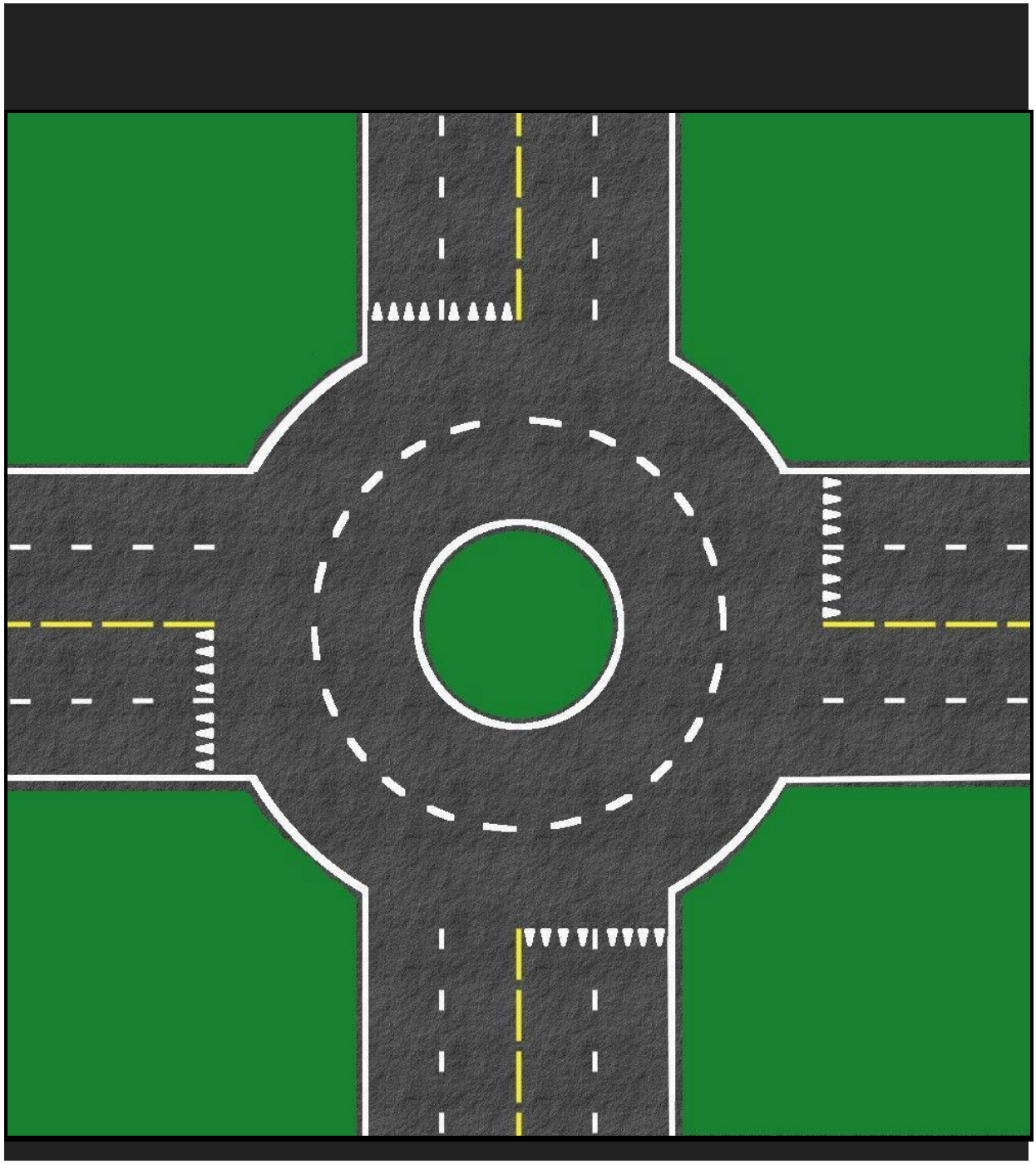
4. Pages (minimum 5)

I recommend exactly five main pages, with multiple routes handling actions within them.



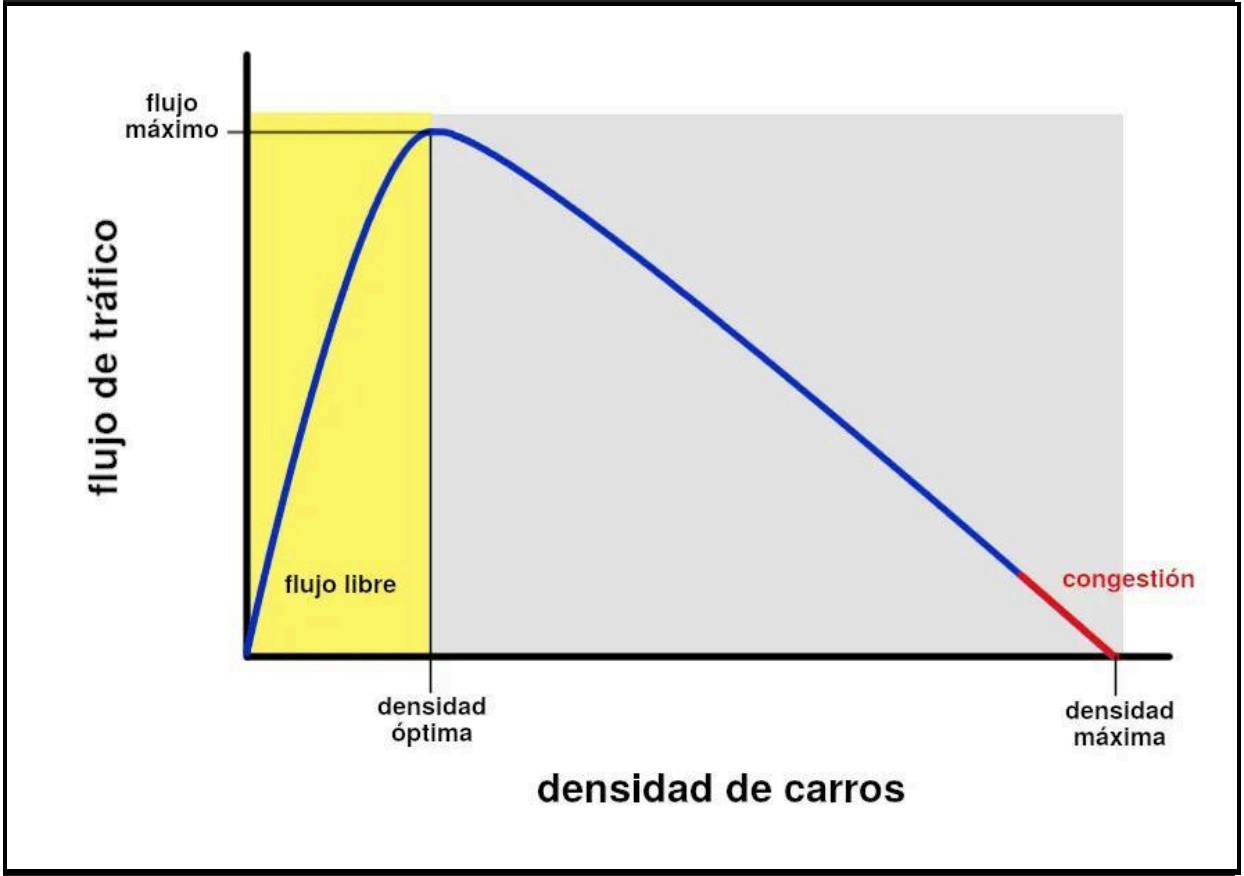
Page 1: Homepage

Introduces the purpose of the program and explains how users can explore traffic flow.



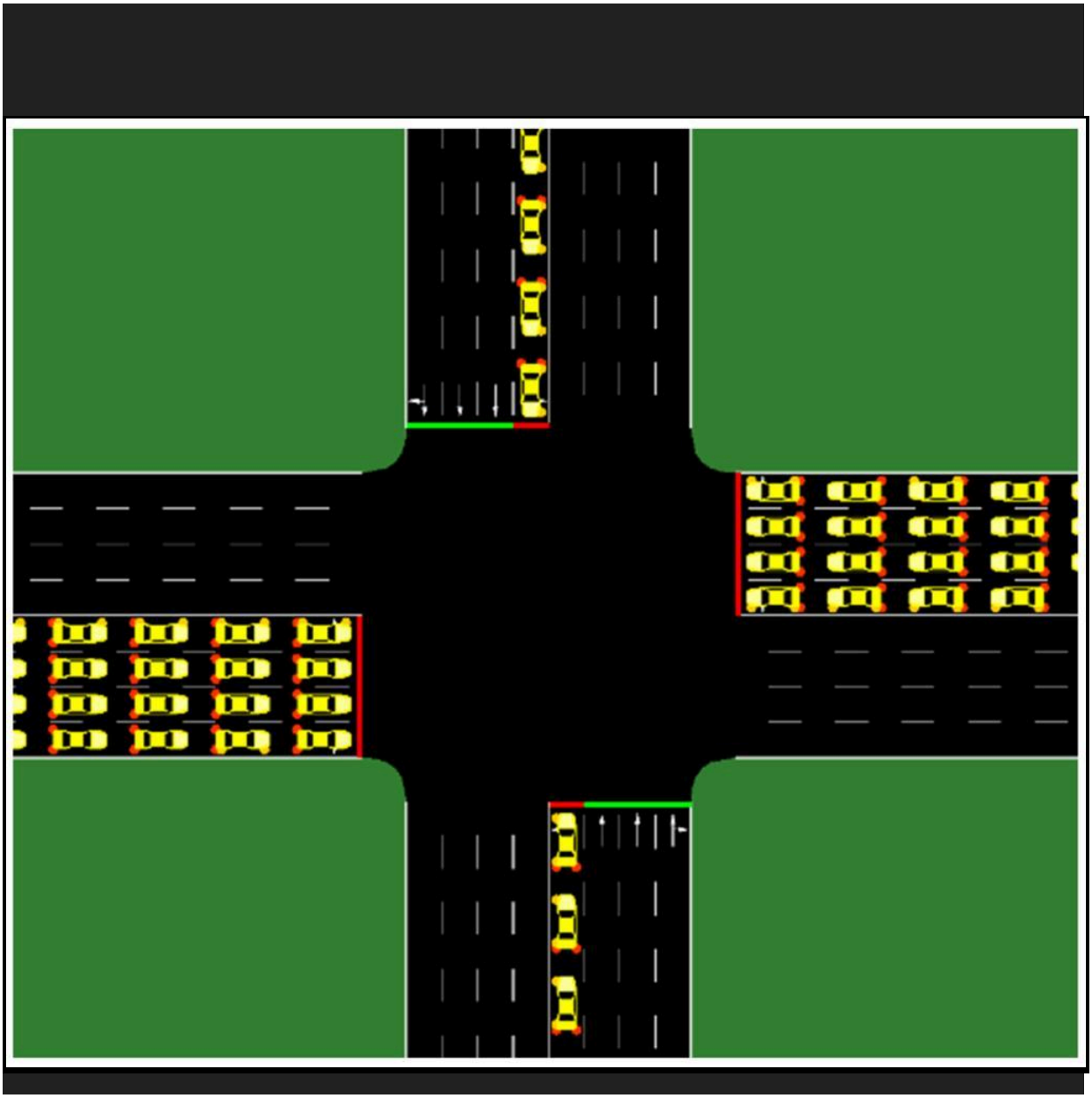
Page 2: Intersection Selection

Users select an intersection design, such as a four-way intersection, roundabout, or T-intersection.



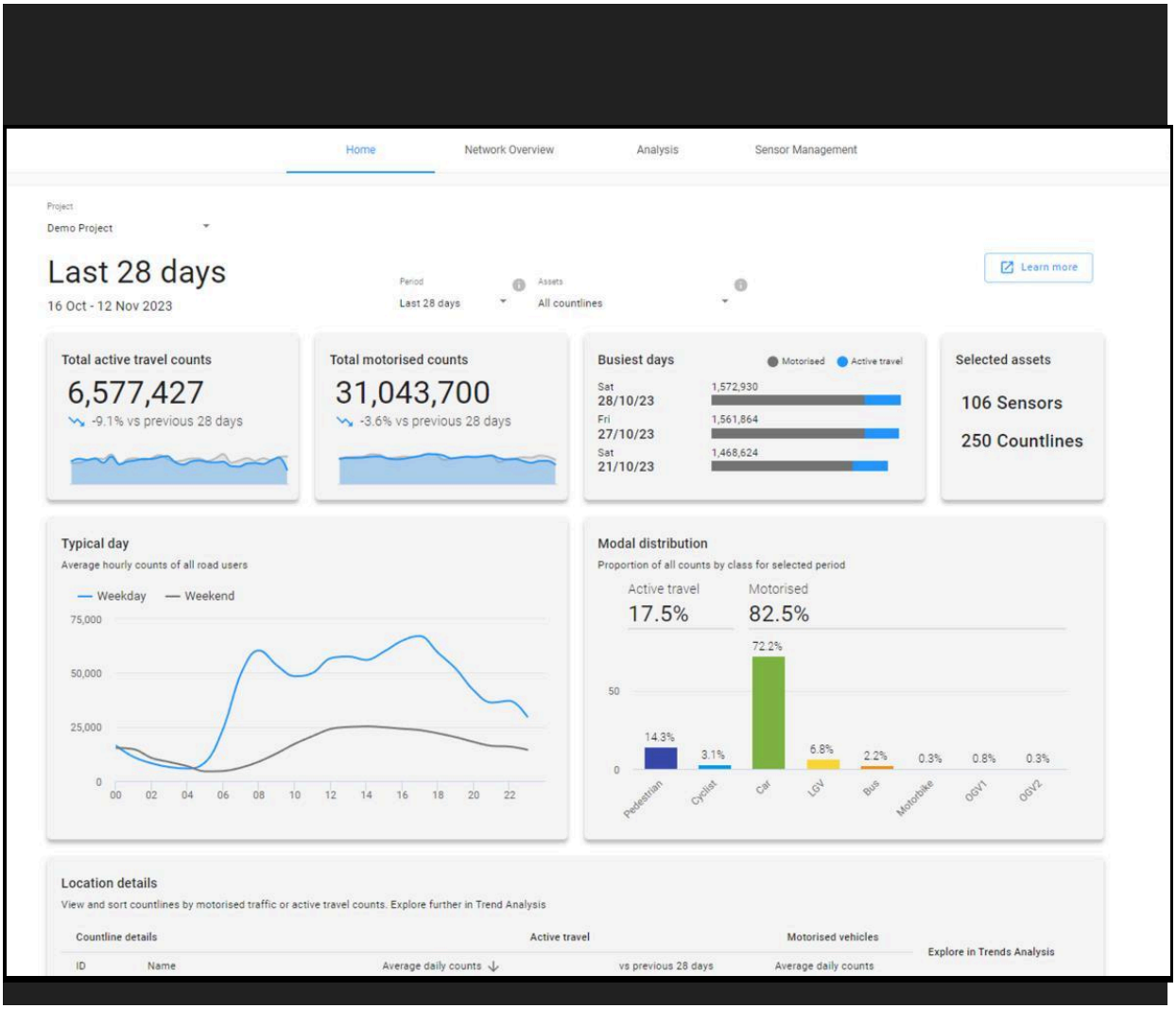
Page 3: Traffic Settings

Users select low, medium, or high traffic density and configure the simulation.



Page 4: Simulation

Displays a top-down visualization of the intersection, vehicle positions, and traffic congestion. Users can advance or reset the simulation.



Page 5: Results

Displays statistics such as vehicles processed, average waiting time, and congestion level. Users can return to settings to test a different scenario.

The images above are visual references, not exact Drafter layouts.

5. User Inputs (minimum 3)

I recommend using three `SelectBox` components because they are easy to implement in Drafter.

Input component	Purpose
`SelectBox` — Intersection Type	Allows users to choose a four-way intersection, roundabout, or T-intersection.
`SelectBox` — Traffic Density	Allows users to select low, medium, or high traffic density.
`SelectBox` — Vehicle Speed	Allows users to select slow, normal, or fast vehicle movement.

These inputs directly affect the simulation.

For example, a user could select:

Example simulation settings

Intersection type

Traffic density

Low

Medium

High

Vehicle speed

Example interface for planning your controls, not a running Drafter simulation.

6. If Statements (minimum 3)

Your program should use conditional statements that meaningfully affect the simulation.

If statement	What it evaluates	Effect on the program
Traffic density	Checks whether traffic density is low, medium, or high.	Determines how many vehicles are generated.
Intersection type	Checks which intersection the user selected.	Determines which intersection layout and movement rules to use.

Vehicle congestion	Checks whether a vehicle needs to wait because its path is blocked.	Stops or slows vehicles, allowing congestion to develop.
Simulation status (optional)	Checks whether the simulation is running.	Determines whether vehicles can advance.

For example, traffic density could be handled using:

```
if state.traffic_density == 1:


    number_of_cars = 5


elif state.traffic_density == 2:


    number_of_cars = 10


else:
```

```
    number_of_cars = 20
```

These values are just starting points. You can adjust them after testing your visualizations.

7. Loop (minimum 1)

Your loop should iterate through the `vehicles` list in your `State` dataclass.

Purpose: Update the positions of every vehicle during each simulation step, allowing the program to represent traffic moving through an intersection.

Example:

```
for vehicle in state.vehicles:
```

```
   if vehicle.waiting == False:


       vehicle.position += vehicle.speed
```

This loop is meaningful because it updates the simulation rather than simply displaying information.

Eventually, you can expand it to check whether each vehicle should stop or slow down based on traffic conditions.

8. How the entire program would work



Select Intersection

Four-way / Roundabout / T-intersection

Configure Traffic

Choose density and vehicle speed

Run Traffic Simulation

Display vehicles, advance positions, track congestion

View Results

Compare vehicles processed and waiting times

9. My recommendation for the project

I think the most important part of your project is the visualization, because your

purpose is to help users understand how different intersection designs affect traffic flow.

However, Drafter is primarily designed for building interactive web pages, not complex animated simulations.

To keep your project manageable, I recommend:

- Starting with three intersection types.
- Using a top-down, 2D diagram of each intersection.
- Representing vehicles as small colored rectangles or symbols.
- Allowing the user to click Advance Simulation to update vehicle positions.
- Tracking simple statistics, such as the number of waiting vehicles and vehicles that successfully pass through the intersection.

For the visualizations, you could use Python to generate updated SVG diagrams and display them in Drafter, provided your course environment supports that approach.

One thing I would change from the original plan is to focus more on traffic density than vehicle speed. Density is the variable most directly connected to your purpose, and

you can calculate vehicle speed based on congestion instead of making users control it.

One question before we start designing the code: Do you want the cars to actually move across the screen like a traffic simulation, or would you be satisfied with a diagram that updates each time the user clicks a button?

That decision will have the biggest impact on how we implement your project in Drafter.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.